

Data Storage and Filesystems

COMPLECS Team

<https://bit.ly/COMPLECS>

<https://github.com/sdsc-complecs>

COMPLECS NSF award #2320934

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

About COMPLECS

COMPLECS (COMPrehensive Learning for end-users to Effectively utilize CyberinfraStructure) is a new training program offered by SDSC that covers the most important ***non-programming*** concepts and skills that you need to effectively use supercomputers. Topics include parallel computing concepts, Linux tools and bash scripting, security, batch and interactive computing, how to get help, and data management.

COMPLECS is supported by NSF [CISE/OAC-2320934](#)



COMPLECS: Data Management Series

Data Storage and File Systems

How to use the data storage and file systems you'll find mounted on high-performance computing systems.

Data Transfer

How to get the data you need for your research to and from high-performance computing systems.

Parallel I/O

How to make effective use of parallel filesystems.





Data Storage and Filesystems

Learning Goals:

- What is a filesystem?
- What are the most common file systems you'll find on HPC systems today?
- How do you choose which file system to use for your work?
- Why is understanding metadata so important to effectively utilizing HPC storage systems?

Prerequisite Knowledge:

- [Intermediate Linux and Shell Scripting](#)

Table of Contents

- **What is a filesystem?**
- **Data storage devices and systems**
- **Files, Directories, and Inodes**
- **Filesystems in High-Performance Computing**
- **Filesystem performance: An AI example**
- **Summary and Conclusion: Best practices**
- **Questions & Answers (30 min)**

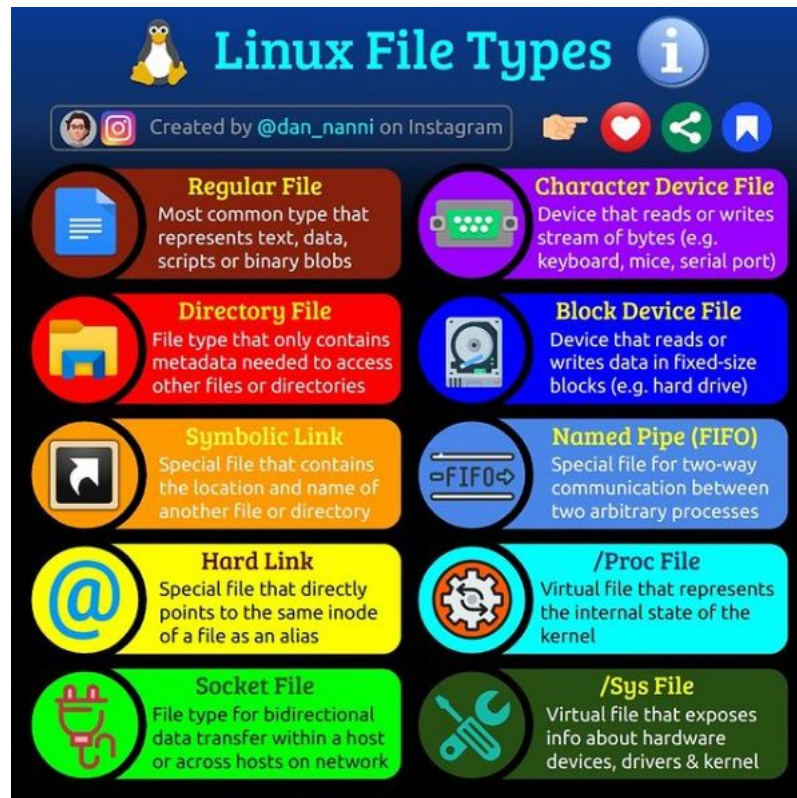
Data Storage

Data storage is the process of recording information (or data) to a *storage medium* such that it is both preserved and retrievable.



Computer File

A computer file is a system resource for recording data on a storage device. Files have different specialized formats to provide encodings for the type of data or information that they are designed to store. On Unix-like operating systems: Everything is a file!



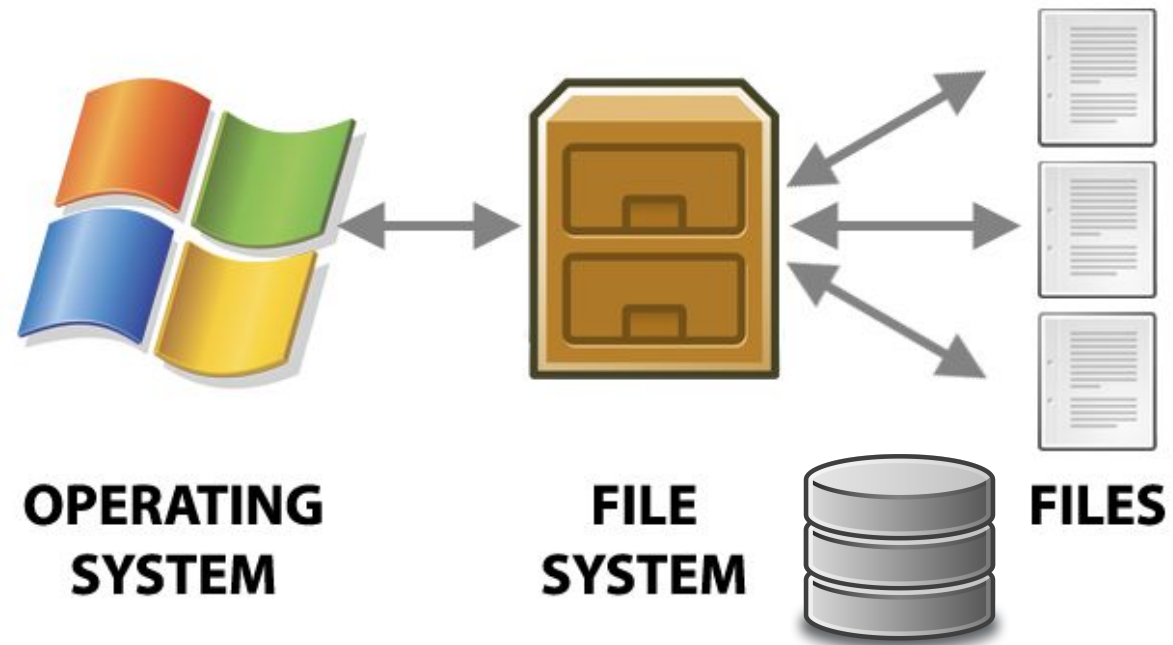
```
mkandes@login01:~  
[mkandes_test@login01]~% cd /  
[mkandes_test@login01]~% ls -lahtr  
total 312K  
drwxr-xr-x  2 root root 4.0K Nov 15 2019 tftpboot  
drwxr-xr-x  2 root root 4.0K Oct 10 2021 srv  
drwxr-xr-x  2 root root 4.0K Oct 10 2021 media  
-rw-r--r--  1 root root  0 May 19 2022 .autorelabel  
lrwxrwxrwx  1 root root  8 Apr  5 2023 sbin -> usr/sbin  
lrwxrwxrwx  1 root root  9 Apr  5 2023 lib64 -> usr/lib64  
lrwxrwxrwx  1 root root  7 Apr  5 2023 lib -> usr/lib  
lrwxrwxrwx  1 root root  7 Apr  5 2023 bin -> usr/bin  
drwxr-xr-x 13 root root 4.0K Apr  7 2023 usr  
drwxr-xr-x  2 root root 4.0K Jun  8 2023 mnt  
drwxr-xr-x  6 root root 4.0K Sep 21 2023 cm  
drwxr-xr-x  7 root root 4.0K May  3 2024 opt  
dr-xr-xr-x  5 root root 4.0K Jul  1 2024 boot  
dr-xr-xr-x 3437 root root  0 May  1 06:30 proc  
dr-xr-xr-x 13 root root  0 May  1 06:30 sys  
drwx----- 2 root root 16K May  1 06:32 lost+found  
-rw-r--r--  1 root root 19K May  1 06:33 aquota.user  
-rw-r--r--  1 root root 16K May  1 06:33 aquota.grp  
dr-xr-xr-x 23 root root 4.0K May  1 06:33 ..  
dr-xr-xr-x 23 root root 4.0K May  1 06:33 .  
drwxr-xr-x  3 root root 4.0K May  1 06:33 scratch  
drwxr-xr-x 24 root root 4.0K May  1 06:34 var  
drwxr-xr-x  8 root root 4.0K May  1 06:34 expanse  
drwxr-xr-x  2 root root  0 May  1 06:34 cvmfs  
drwxr-xr-x 147 root root 12K May  1 07:11 etc  
drwxr-xr-x 21 root root 3.8K May  4 08:11 dev  
dr-xr-xr-x  6 root root 4.0K May  6 08:36 root  
drwxr-xr-x 37 root root 1.3K May 15 18:32 run  
drwxr-xr-x 2514 root root  0 May 15 18:58 home  
drwxrwxrwt 458 root root 188K May 15 18:58 tmp
```

File System

A file system (also often written as filesystem) provides an operating system with the ability to create, organize, and manage access to files on storage devices.

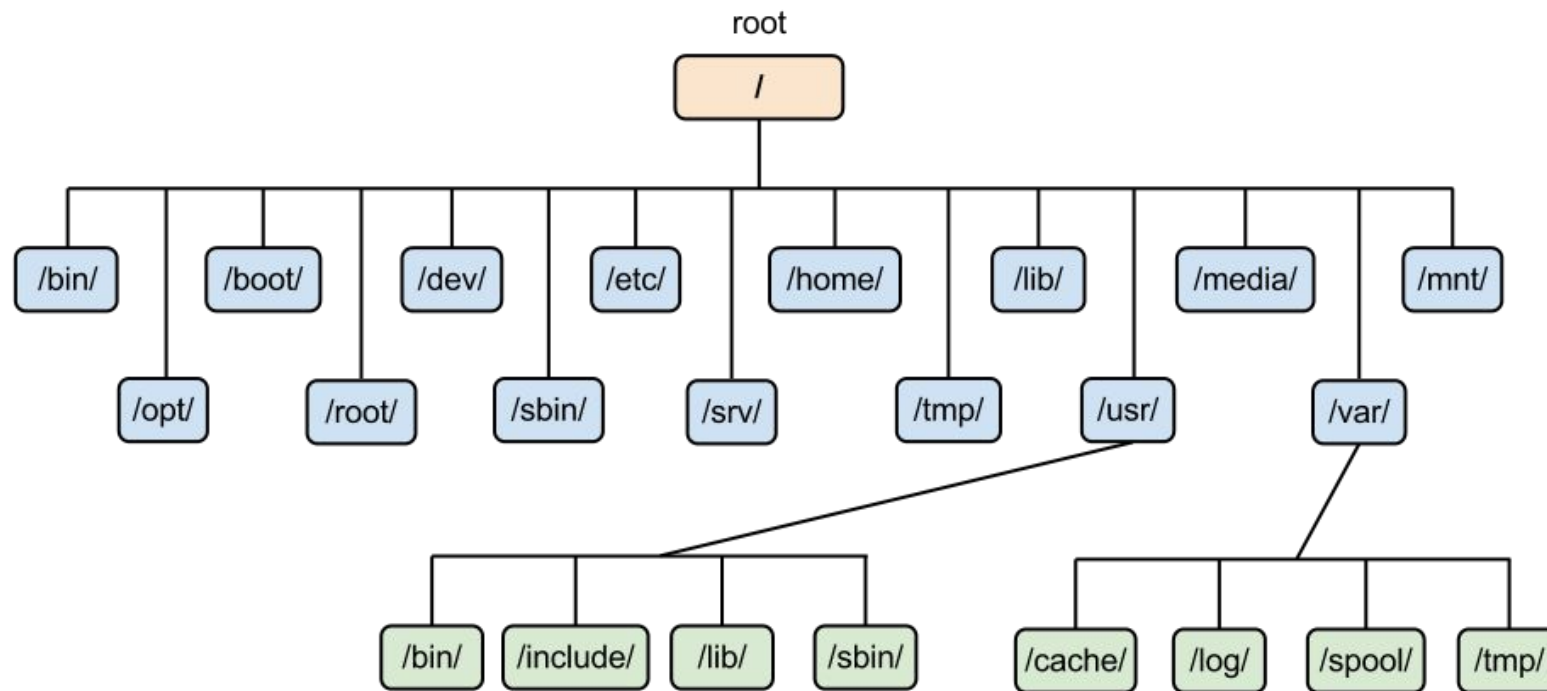
All filesystems support a set of standard operations that can be performed on a file:

- Create a new file
- Open a file
- Read data from a file
- Write data to a file
- Close a file
- Delete a file
- Change access permissions and/or attributes of a file



Filesystem Hierarchy Standard

The [Filesystem Hierarchy Standard \(FHS\)](#) describes the conventions used to organize the layout of standard directories and files in Unix-like operating systems such as Linux.



```
mkandes@login02:~  
[mkandes@login02 ~]$ tree -L 1 -d /  
/  
├── bin -> usr/bin  
├── boot  
├── cm  
├── cvmfs  
├── dev  
├── etc  
├── expance  
├── home  
├── lib -> usr/lib  
├── lib64 -> usr/lib64  
├── lost+found  
├── media  
├── mnt  
├── opt  
├── proc  
├── root  
├── run  
├── sbin -> usr/sbin  
├── scratch  
├── srv  
├── sys  
├── tftpboot  
├── tmp  
├── usr  
└── var  
  
25 directories  
[mkandes@login02 ~]$
```

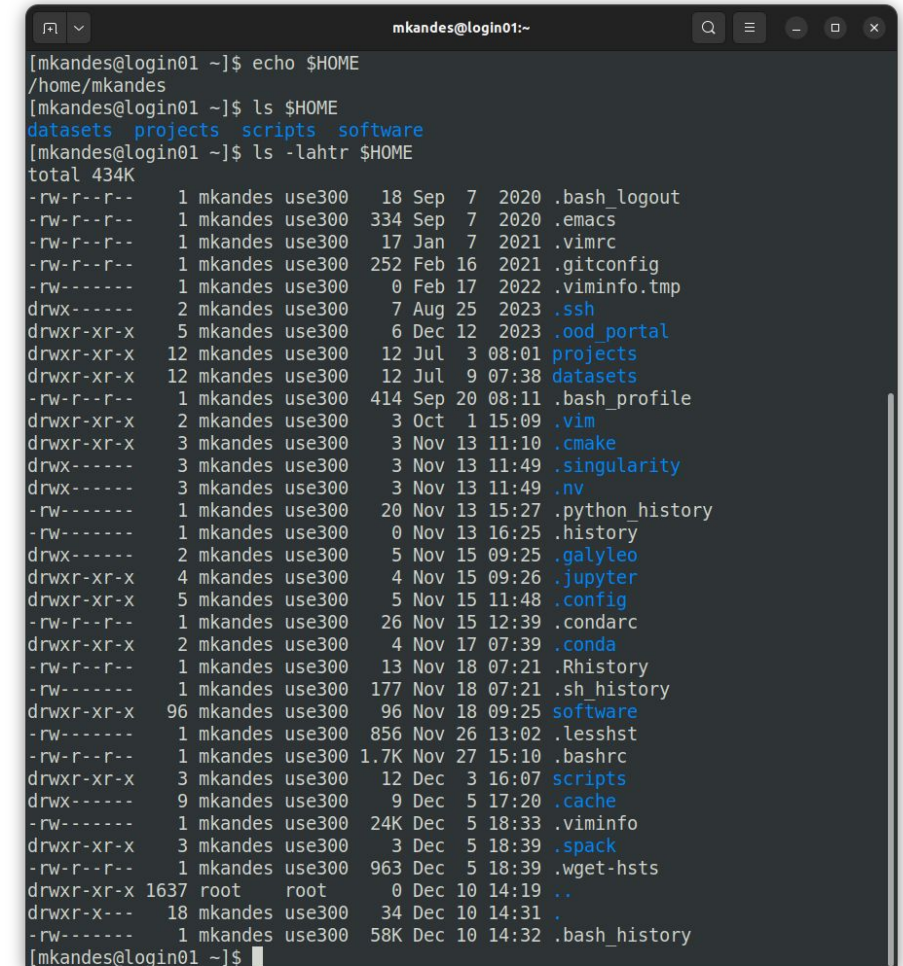
The most important directory for you is **/home**. This is where your **\$HOME** directory resides.

\$HOME

A [\\$HOME](#) directory on a shared multi-user system contains the directories and files owned by a given [user](#) of the system.

On HPC systems, your \$HOME directory:

- is your [working directory](#) on [login](#);
- is where you will install software, create and submit batch job scripts, and store input and output files for your jobs;
- has a limited amount of storage available; and
- will support a limited number of simultaneous read and/or write operations.

A terminal window titled 'mkandes@login01:~' showing the contents of the \$HOME directory. The user runs 'echo \$HOME' which returns '/home/mkandes'. Then they run 'ls \$HOME' which lists 'datasets', 'projects', 'scripts', and 'software'. Finally, they run 'ls -lahtr \$HOME' which shows a detailed listing of files and directories in the home directory, including hidden files like '.bash_logout', '.emacs', '.vimrc', '.gitconfig', '.viminfo.tmp', '.ssh', '.ood_portal', 'projects', 'datasets', '.bash_profile', '.vim', '.cmake', '.singularity', '.nv', '.python_history', '.history', '.galyleo', '.jupyter', '.config', '.condarc', '.conda', '.Rhistory', '.sh_history', 'software', '.lessht', '.bashrc', 'scripts', '.cache', '.viminfo', '.spack', '.wget-hsts', and '.bash_history'. The total size of the directory is 434K.

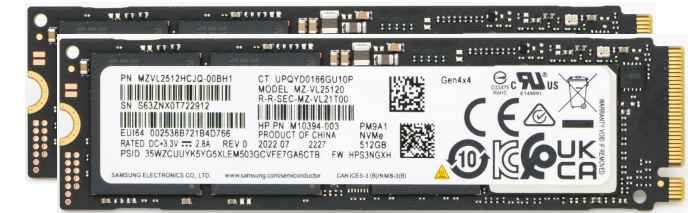
```
[mkandes@login01 ~]$ echo $HOME
/home/mkandes
[mkandes@login01 ~]$ ls $HOME
datasets  projects  scripts  software
[mkandes@login01 ~]$ ls -lahtr $HOME
total 434K
-rw-r--r--  1 mkandes  use300   18 Sep  7  2020 .bash_logout
-rw-r--r--  1 mkandes  use300  334 Sep  7  2020 .emacs
-rw-r--r--  1 mkandes  use300   17 Jan  7  2021 .vimrc
-rw-r--r--  1 mkandes  use300  252 Feb 16  2021 .gitconfig
-rw-r--r--  1 mkandes  use300    0 Feb 17  2022 .viminfo.tmp
drwx-----  2 mkandes  use300    7 Aug 25  2023 .ssh
drwxr-xr-x  5 mkandes  use300    6 Dec 12  2023 .ood_portal
drwxr-xr-x 12 mkandes  use300   12 Jul  3 08:01 projects
drwxr-xr-x 12 mkandes  use300   12 Jul  9 07:38 datasets
-rw-r--r--  1 mkandes  use300  414 Sep 20 08:11 .bash_profile
drwxr-xr-x  2 mkandes  use300    3 Oct  1 15:09 .vim
drwxr-xr-x  3 mkandes  use300    3 Nov 13 11:10 .cmake
drwx-----  3 mkandes  use300    3 Nov 13 11:49 .singularity
drwx-----  3 mkandes  use300    3 Nov 13 11:49 .nv
-rw-r--r--  1 mkandes  use300   20 Nov 13 15:27 .python_history
-rw-r--r--  1 mkandes  use300    0 Nov 13 16:25 .history
drwx-----  2 mkandes  use300    5 Nov 15 09:25 .galyleo
drwxr-xr-x  4 mkandes  use300    4 Nov 15 09:26 .jupyter
drwxr-xr-x  5 mkandes  use300    5 Nov 15 11:48 .config
-rw-r--r--  1 mkandes  use300   26 Nov 15 12:39 .condarc
drwxr-xr-x  2 mkandes  use300    4 Nov 17 07:39 .conda
-rw-r--r--  1 mkandes  use300   13 Nov 18 07:21 .Rhistory
-rw-r--r--  1 mkandes  use300  177 Nov 18 07:21 .sh_history
drwxr-xr-x 96 mkandes  use300   96 Nov 18 09:25 software
-rw-r--r--  1 mkandes  use300  856 Nov 26 13:02 .lessht
-rw-r--r--  1 mkandes  use300  1.7K Nov 27 15:10 .bashrc
drwxr-xr-x  3 mkandes  use300   12 Dec  3 16:07 scripts
drwx-----  9 mkandes  use300    9 Dec  5 17:20 .cache
-rw-r--r--  1 mkandes  use300  24K Dec  5 18:33 .viminfo
drwxr-xr-x  3 mkandes  use300    3 Dec  5 18:39 .spack
-rw-r--r--  1 mkandes  use300  963 Dec  5 18:39 .wget-hsts
drwxr-xr-x 1637 root    root    0 Dec 10 14:19 ..
drwxr-xr-x  18 mkandes  use300   34 Dec 10 14:31 .
-rw-r--r--  1 mkandes  use300  58K Dec 10 14:32 .bash_history
[mkandes@login01 ~]$
```


Local vs. Distributed Filesystems

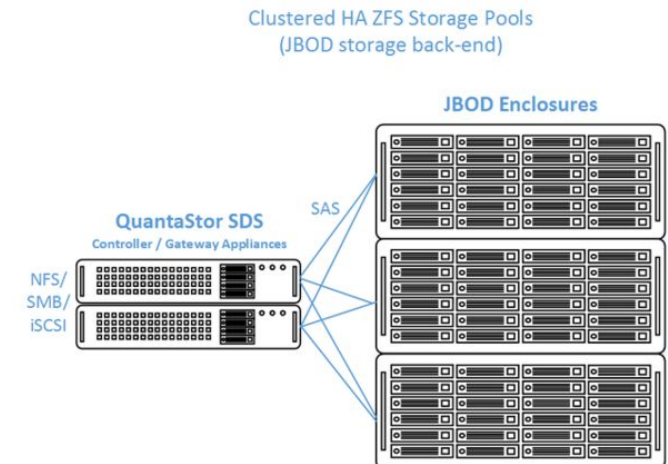
There are many different types of filesystems. And every type of filesystem has its own advantages and limitations, which will determine **how** and **where** you will want to store different **types** of data at any given **time**. However, there are two general classes of filesystems you will interact with on any HPC system.

A **local file system** provides the ability to interact with files on a storage device **attached to the same computer**. In other words, files are *isolated* and *not accessible from other computers*.

A **distributed file system** provides a service to store and **share** files across multiple storage devices **over a network** protocol. In general, however, they allow files to be accessed by many nodes in a cluster using the same interfaces and semantics as a local file system.



RAID



HA

df - report file system disk space usage

df (an abbreviation for *disk free*) is a standard Unix command used to display the amount of available disk space for file systems.

```
mkandes@login01:~$ df -Th | head -n 20
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
devtmpfs	devtmpfs	63G	0	63G	0%	/dev
tmpfs	tmpfs	63G	11M	63G	1%	/run
/dev/sda2	ext4	32G	24G	6.0G	80%	/
tmpfs	tmpfs	63G	14G	50G	22%	/dev/shm
tmpfs	tmpfs	63G	0	63G	0%	/sys/fs/cgroup
/dev/sda1	vfat	100M	0	100M	0%	/boot/efi
/dev/sda4	ext4	32G	13G	17G	43%	/tmp
/dev/sdb1	ext4	879G	44K	834G	1%	/scratch
master:/home	nfs	140G	98G	43G	70%	/exppanse/nfs/mgr1/home
ps-071.sdsc.edu:/ps-data/community-sw	nfs	1.0T	324G	701G	32%	/exppanse/community
10.22.100.114:/pool4/home	nfs	207T	22T	185T	11%	/exppanse/nfs/home4
10.22.100.113:/pool3/home	nfs	199T	22T	177T	12%	/exppanse/nfs/home3
10.22.100.112:/pool2/home	nfs	210T	28T	183T	13%	/exppanse/nfs/home2
10.22.100.111:/pool1/home	nfs	214T	21T	193T	10%	/exppanse/nfs/home1
10.21.0.21:6789,10.21.11.7:6789,10.21.11.8:6789:/	ceph	1.6T	1.2T	467G	71%	/cm/shared
10.22.101.201:/cw3e	nfs	82T	1.0M	82T	1%	/exppanse/nfs/cw3e
192.168.43.5:6789,192.168.43.6:6789:/	ceph	2.0P	159T	1.9P	8%	/exppanse/ceph
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/projects	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/projects
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/scratch	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/scratch

```
[mkandes@login01 ~]$
```



Disk Quota

A [disk quota](#) (or simply, quota) is a limit set by a system administrator that restricts certain aspects of file system usage on modern operating systems. In general, quotas are used to *allocate limited disk space* in a reasonable way.

So, how do you find out what your quota is?

Unfortunately, this will vary by filesystem. Some filesystems have standard command-line tools, while some HPC administrators have their own custom command-line tools and/or storage usage reporting for users.

The best place to start is to review the storage section of your HPC system's user guide. e.g., https://www.sdsc.edu/support/user_guides/expanse.html#storage

Note, however, there is one standard command to check how much space you are currently consuming on a given filesystem ...

du - estimate file space usage

du (an abbreviation for *disk usage*) is a standard Unix program used to estimate file space usage — space used under a particular directory or files on a file system.

```
mkandes@login02:~  
[mkandes@login02 ~]$ du -h $HOME  
2.8M    /home/mkandes/software/mpi4py/releases  
2.8M    /home/mkandes/software/mpi4py  
14M     /home/mkandes/software/iqtree2/releases  
14M     /home/mkandes/software/iqtree2  
217M    /home/mkandes/software/gcc/releases  
217M    /home/mkandes/software/gcc  
12M     /home/mkandes/software/subversion/releases  
12M     /home/mkandes/software/subversion  
1.6M    /home/mkandes/software/pkgconf/releases  
1.6M    /home/mkandes/software/pkgconf  
19M     /home/mkandes/software/hdf5/releases  
19M     /home/mkandes/software/hdf5  
3.2M    /home/mkandes/software/bamutil/releases  
3.2M    /home/mkandes/software/bamutil  
1.6M    /home/mkandes/software/git-lfs/releases  
1.6M    /home/mkandes/software/git-lfs  
4.5G    /home/mkandes/software/spark  
24M     /home/mkandes/software/lapack/releases  
24M     /home/mkandes/software/lapack  
4.7M    /home/mkandes/software/mpmath/releases  
4.7M    /home/mkandes/software/mpmath  
263M    /home/mkandes/software/boost/releases  
263M    /home/mkandes/software/boost
```

This command can take quite some time to run depending on the total number of directories and files *AND* on the *type of filesystem* the top-level directory is located on.



Table of Contents

- What is a filesystem?
- **Data storage devices and systems**
- Files, Directories, and Inodes
- Filesystems in High-Performance Computing
- Filesystem performance: An AI example
- Summary and Conclusion: Best practices
- Questions & Answers (30 min)

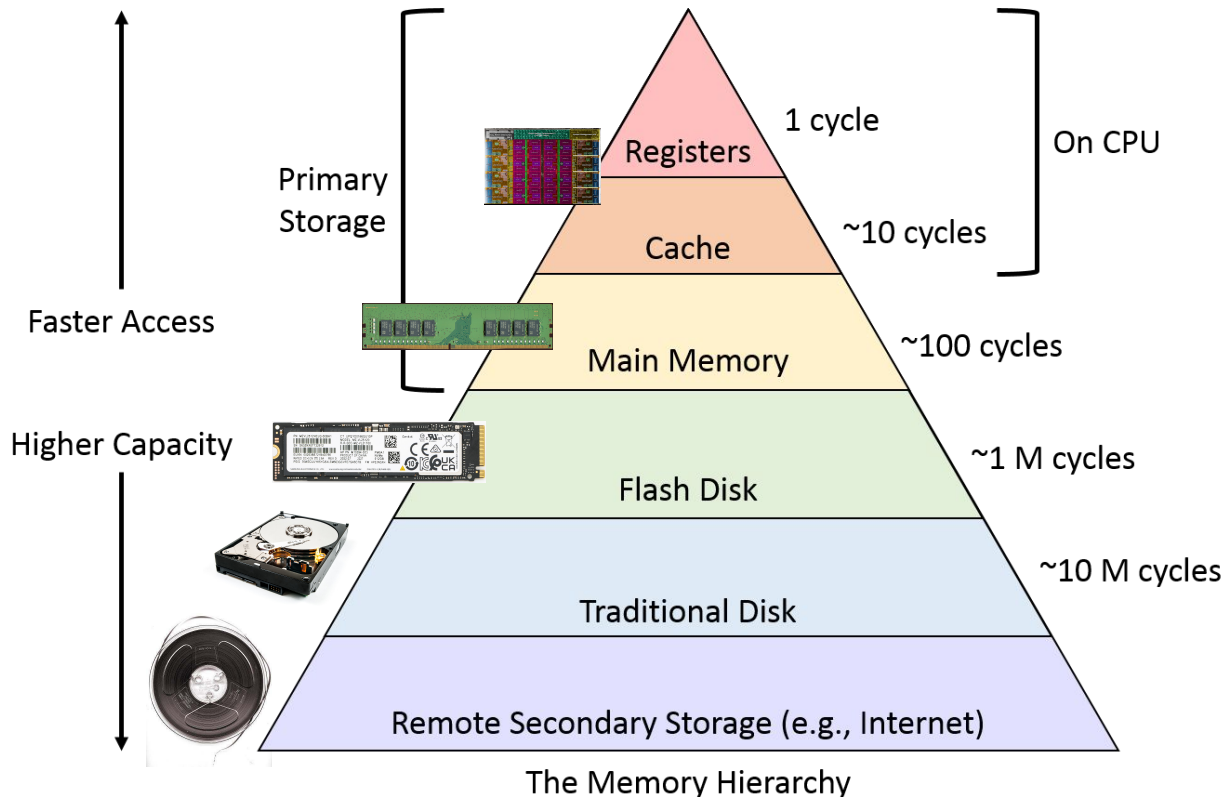
Computer Data Storage Systems (or Devices)

A computer data storage system (or device) consists of the physical hardware components, recording media, and other technologies that are used to retain digital information (or data).

The memory hierarchy separates computer storage based on response time, which is typically referred to as latency.

Four key measures of a storage system are:

- **Capacity** - How much data can it hold?
- **Performance** - How quickly and efficiently can it read and write data?
- **Reliability** - How long can it maintain data integrity and availability over time?
- **Scalability** - How well can it accommodate growth in data volumes and compute workloads?



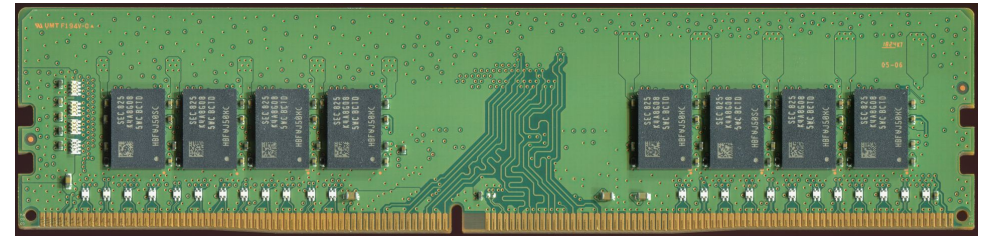
Memory

Memory stores information, such as data and programs, for immediate use in a computer's central processing unit(s) (or some other computation device — e.g. a GPU). Memory is often synonymous with the terms RAM, main memory, or primary storage.

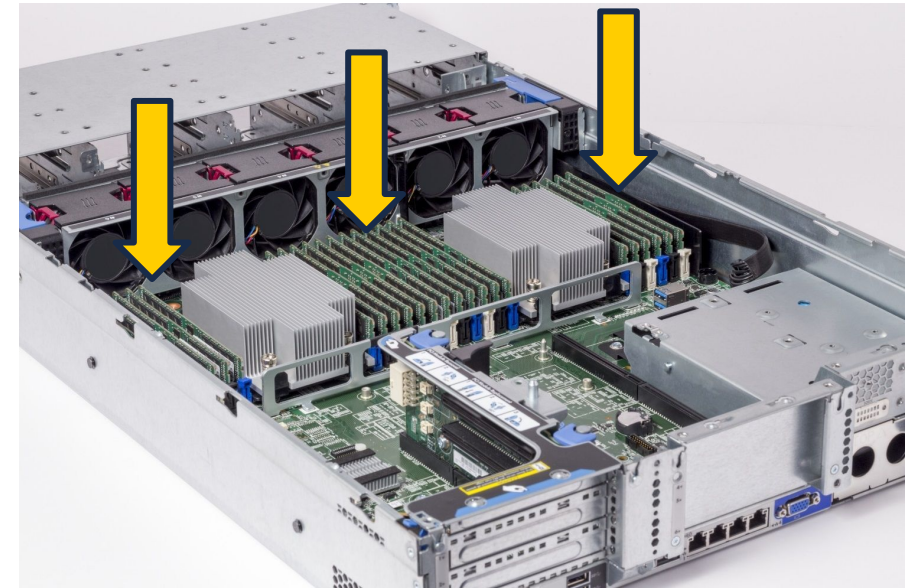
Memory modules on HPC systems are almost always error-correcting, which increases their reliability while in use.

Many applications can control how much data they store in memory. However, this is often because the data is larger than the memory available on a compute node — e.g., see out-of-core algorithms.

Memory is also where your *fastest filesystem* resides — e.g., see RAM disk.



DDR4



free - display amount of memory in the system

free is a UNIX command which displays the total amount of free and used physical and swap memory in the system, as well as the shared memory and buffers used by the kernel.

```
mkandes@login02:~$ free
              total        used        free      shared  buff/cache   available
Mem:      130911100      57384536      21404560      2526804      52122004      69379844
Swap:      12582908      12579816           3092

mkandes@login02:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          124Gi         54Gi         18Gi         2.4Gi         51Gi         65Gi
Swap:          11Gi         11Gi         3.0Mi

mkandes@login02:~$
```

```
mkandes@login02:~$ srun --partition=debug --account=use300 --nodes=1 --ntasks
-per-node=1 --cpus-per-task=8 --mem=16G --time=00:30:00 --pty --wait=0 /bin/bash
srun: job 35758722 queued and waiting for resources
srun: job 35758722 has been allocated resources
mkandes@exp-9-55:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          251Gi         7.7Gi        242Gi         119Mi         755Mi        242Gi
Swap:           0B           0B           0B

mkandes@exp-9-55:~$
```

```
mkandes@login02:~$ srun --partition=gpu-debug --account=use300 --nodes=1 --nt
asks-per-node=1 --cpus-per-task=10 --mem=92G --gpus=1 --time=00:30:00 --pty --wa
it=0 /bin/bash
srun: job 35758835 queued and waiting for resources
srun: job 35758835 has been allocated resources
mkandes@exp-7-59:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          376Gi         2.8Gi        373Gi         119Mi         459Mi        371Gi
Swap:           0B           0B           0B

mkandes@exp-7-59:~$
```

A few things to remember on HPC systems:

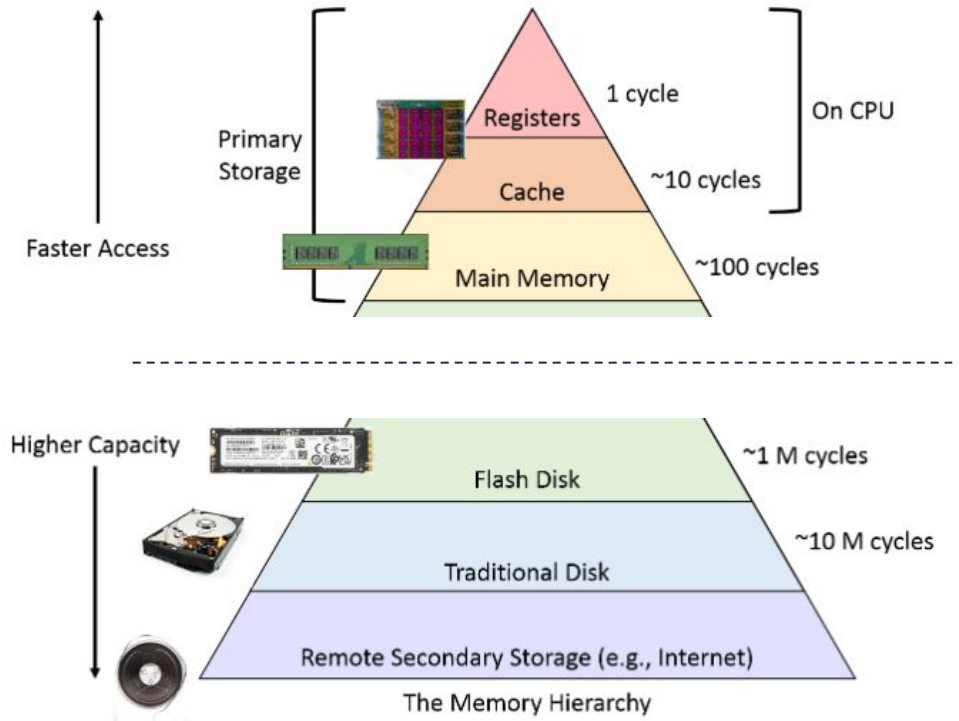
- Different node types will have different amounts of memory.
- The **free** memory shown is likely not the amount available for your batch job, unless it has exclusive access to the compute node.
- Only CPU memory availability and usage is reported. Other compute devices like GPUs have their own on-board memory hierarchy, and their own command-line tools to query this type of information. e.g., [nvidia-smi](#)

Volatile vs. Non-Volatile Memory (or Storage)

There are many different types of memory and storage devices. However, there are two general classes of storage devices you will interact with on any computing system.

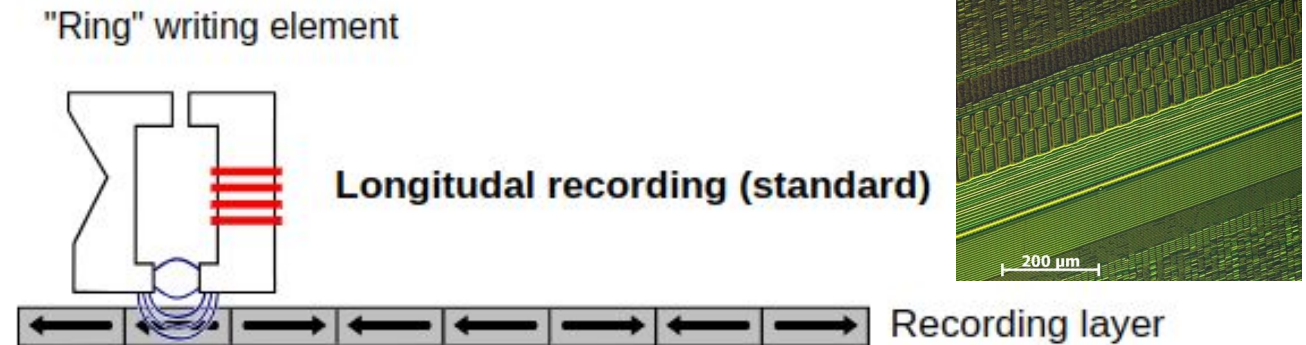
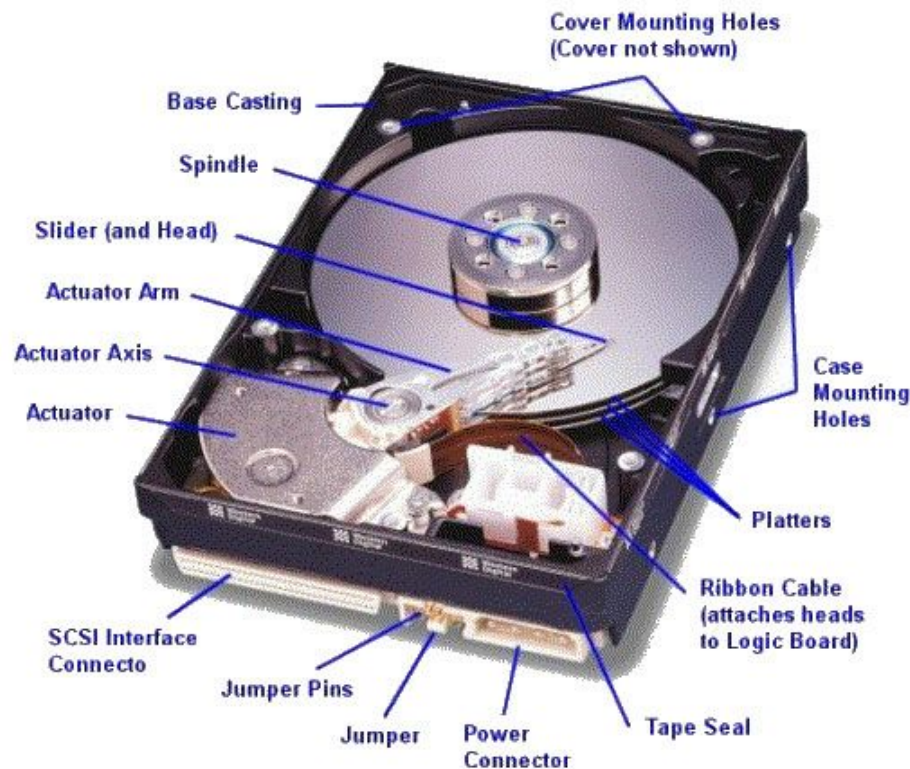
Volatile memory **requires power** to maintain stored data; it retains its contents while powered on *but when the power is interrupted, the stored data is quickly lost*. It is used as primary storage.

Non-volatile memory (or storage) can retain stored data even **after power is removed**. It is used for secondary (or long-term persistent) storage.



Hard Disk Drives (HDDs)

A hard disk drive (HDD) is an electro-mechanical data storage device that stores and retrieves digital data using magnetic storage with one or more rigid rapidly rotating platters coated with magnetic material.



While HDDs are not used as node-local storage on most HPC compute nodes these days, they are still widely deployed as high-capacity storage devices as part of a distributed filesystem. i.e., the performance of a distributed filesystem may also be limited due to the mechanical properties of the type of disks in use.

Solid State Drives (SSDs)

A solid-state drive (SSD) is a type of solid-state storage device that uses only integrated circuits to store data persistently. *No moving parts!*



Like your personal computer, SSDs are used for node-local storage on most HPC compute nodes these days. e.g., at least one SSD in every node will host the files required to run its operating system.

The next most important use case is to serve as a node-local **/scratch** disk, where you can temporarily read in and/or write files to while your batch job is running.

Also used to create a fast tier of storage in a multi-tier distributed file system.

Magnetic Tape

Magnetic tape is a medium for magnetic storage made of a thin, magnetizable coating on a long, narrow strip of plastic film. It is still widely used as a part of modern data storage systems. However, they are used mostly for archival and backup use cases.



See Ranch at TACC!

Table of Contents

- What is a filesystem?
- Data storage devices and systems
- **Files, Directories, and Inodes**
- Filesystems in High-Performance Computing
- Filesystem performance: An AI example
- Summary and Conclusion: Best practices
- Questions & Answers (30 min)

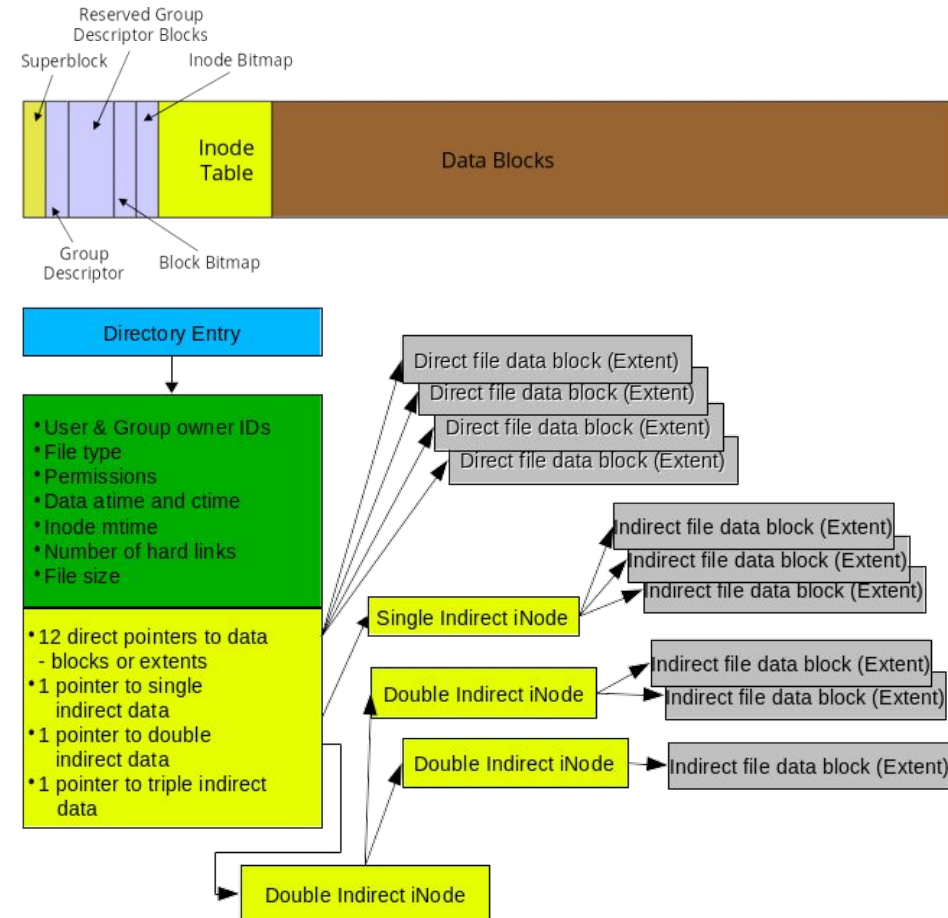
Elements of a POSIX filesystem

POSIX filesystems are organized as directories containing files. However, there are also more fundamental elements in POSIX compliant filesystems like inodes and data blocks.

All filesystems support a set of standard operations that can be performed on a file:

- Create a new file
- Change the access permissions and/or attributes of a file
- Open a file
- Read data from a file
- Write data to a file
- Delete a file
- Close a file

All of the filesystems you will likely interact with on an HPC system are [POSIX](#)-based.



Block

A [block](#), sometimes also called a *physical record*, is a sequence of [bytes](#) or [bits](#), usually containing some whole number of records, having a maximum length; a *block size*.

POSIX filesystems are treated as a sequence of bytes. As such, internally the data stored within a file is a logical sequence of filesystem blocks.

- Each block is a fixed number of bytes (usually 4 KiB). Last block(s) need not be filled.
- All the bytes are sequential within a block. However, the blocks might not reside sequentially on the disk within a file.
- Underlying storage systems are usually organized as blocks, and ideally filesystem blocks are aligned with the storage system's blocks.

Inode

An [inode \(or index node\)](#) is a data structure in a POSIX filesystem that describes a filesystem object such as a file or a directory. Each inode stores the attributes and disk block *locations* of the object's data. It provides the [metadata](#) about a filesystem object.

Inodes provide metadata such as:

- a timestamp of when the *inode* was last modified ([ctime](#))
- a timestamp of when the *file's contents* were last modified ([mtime](#))
- a timestamp of when file was last accessed ([atime](#))
- the numeric user id ([UID](#)) of the user that is the owner of the data
- the numeric group id ([GID](#)) of the group that is the owner of the backed data
- the POSIX mode bits (often called permissions or permission bits)

Directory

A [directory](#) is a special filesystem object that which contains references to files, and other directories. Each POSIX filesystem may have its own implementation of a directory.

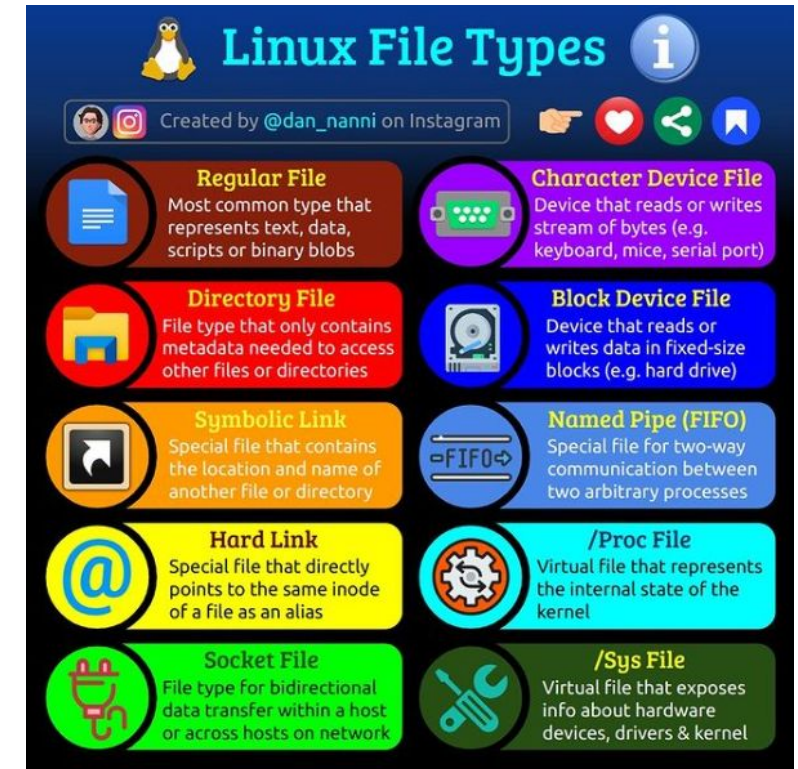
However, common among all filesystems are:

- each directory should at least have a logical name along with the number of the inodes that contains the metadata backing up the entry
- the name can be any string made of any characters with the exception of "/" character and of the "null" character; the length of the string is limited, but the actual length depends the filesystem implementation
- the name is the primary key – no two entries can have the same path

File

A file is a set of blocks used to store data.

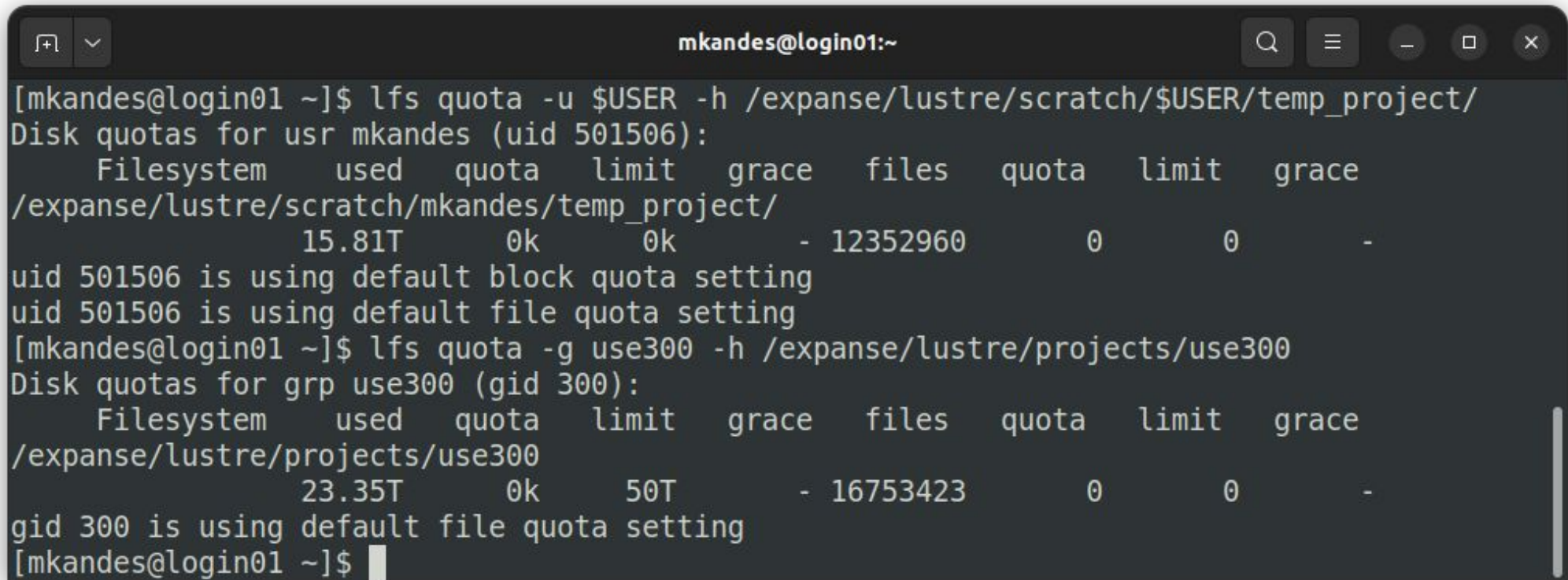
- Files do not have a name.
- They are a set of blocks that can be either directly referenced from the inode or indirectly referenced from another data block.
- The name of the file is stored inside a directory as a directory entry.



Everything is a file!

lfs - client utility for Lustre filesystems

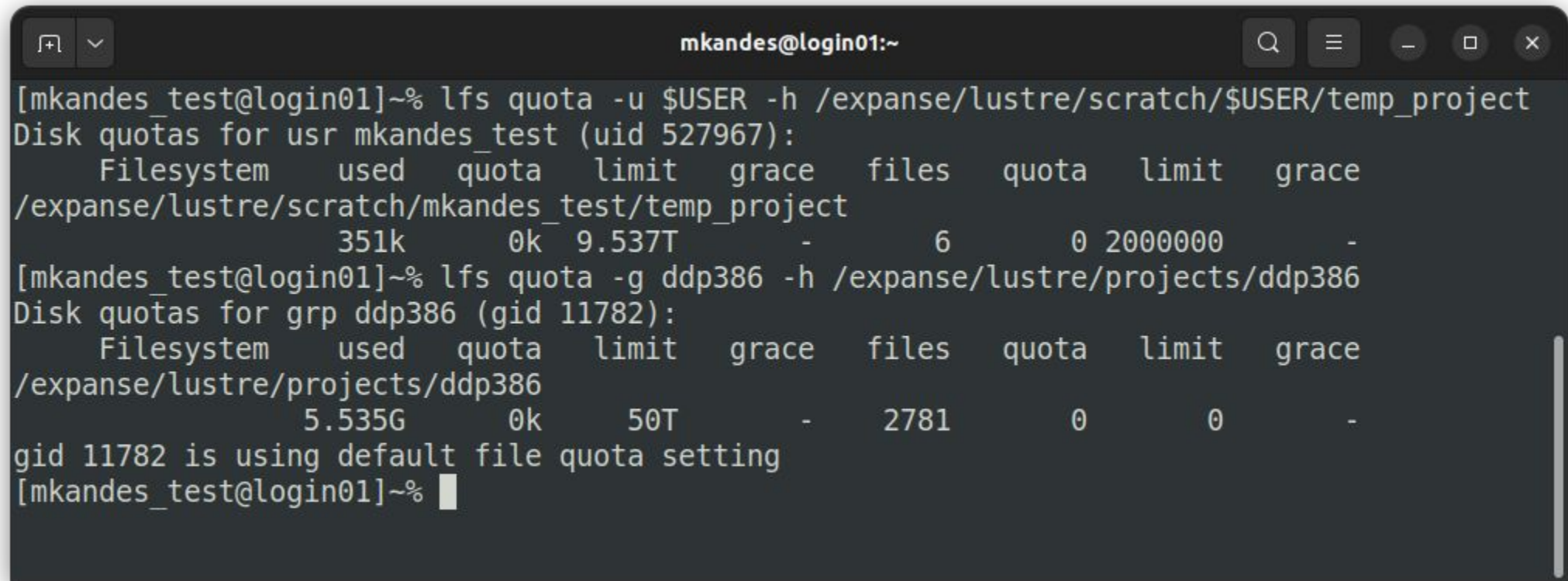
lfs is a command-line utility for Lustre filesystems. It can be used to create a new file with a specific striping pattern, determine the default striping pattern, gather the extended attributes (object numbers and location) for a specific file. You can also use it to check your quota(s).



```
mkandes@login01:~  
[mkandes@login01 ~]$ lfs quota -u $USER -h /expanse/lustre/scratch/$USER/temp_project/  
Disk quotas for usr mkandes (uid 501506):  
    Filesystem      used   quota  limit  grace  files   quota  limit  grace  
/expanse/lustre/scratch/mkandes/temp_project/  
                15.81T    0k     0k      - 12352960      0      0      -  
uid 501506 is using default block quota setting  
uid 501506 is using default file quota setting  
[mkandes@login01 ~]$ lfs quota -g use300 -h /expanse/lustre/projects/use300  
Disk quotas for grp use300 (gid 300):  
    Filesystem      used   quota  limit  grace  files   quota  limit  grace  
/expanse/lustre/projects/use300  
                23.35T    0k    50T      - 16753423      0      0      -  
gid 300 is using default file quota setting  
[mkandes@login01 ~]$
```


lfs - client utility for Lustre filesystems

lfs is a command-line utility for Lustre filesystems. It can be used to create a new file with a specific striping pattern, determine the default striping pattern, gather the extended attributes (object numbers and location) for a specific file. You can also use it to check your quota(s).



```
mkandes@login01:~  
[mkandes_test@login01]~% lfs quota -u $USER -h /expanse/lustre/scratch/$USER/temp_project  
Disk quotas for usr mkandes_test (uid 527967):  
    Filesystem      used      quota      limit      grace      files      quota      limit      grace  
/expanse/lustre/scratch/mkandes_test/temp_project  
                        351k       0k  9.537T          -          6          0 2000000          -  
[mkandes_test@login01]~% lfs quota -g ddp386 -h /expanse/lustre/projects/ddp386  
Disk quotas for grp ddp386 (gid 11782):  
    Filesystem      used      quota      limit      grace      files      quota      limit      grace  
/expanse/lustre/projects/ddp386  
                        5.535G       0k    50T          -      2781          0          0          -  
gid 11782 is using default file quota setting  
[mkandes_test@login01]~% █
```

Table of Contents

- What is a filesystem?
- Data storage devices and systems
- Files, Directories, and Inodes
- **Filesystems in High-Performance Computing**
- Filesystem performance: An AI example
- Summary and Conclusion: Best practices
- Questions & Answers (30 min)

Unit	System Layout	Performance Storage	Object Storage
	12PB Luster	3PB Cold	



HPC System Architecture: Conceptual Model

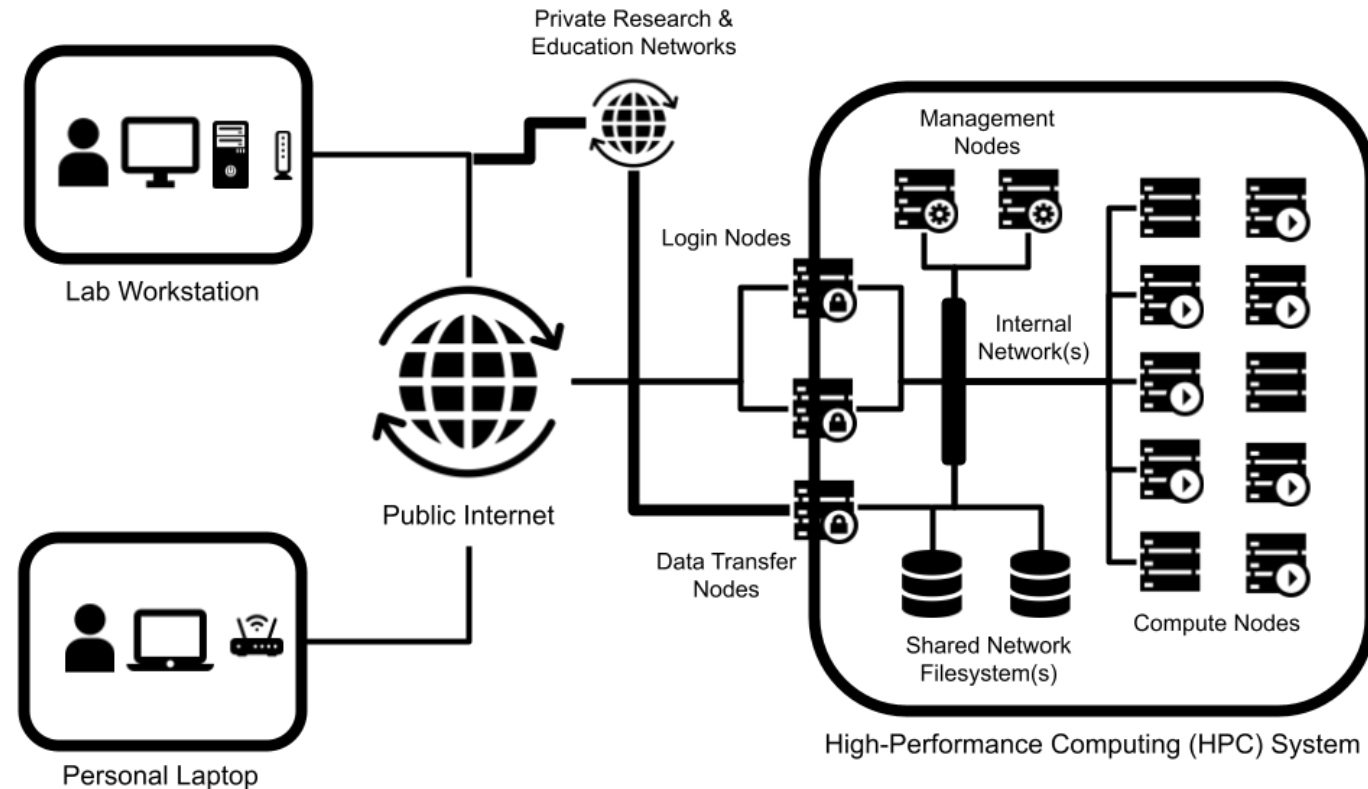
Login node(s): Provide remote access to an HPC system; use only for simple tasks such as editing files, limited data transfers to and from the system, and batch job submission

Compute nodes: Run computational workloads: simulations, data analysis and visualization

Internal Network(s): Provide high-bandwidth, low-latency communication between compute nodes; access to shared (parallel) filesystems; system management

Shared Network Filesystem(s): Provide input/output (I/O) access to data storage systems from any compute node

Data Transfer Node(s) : Deployed and configured specifically for transferring data over networks, usually between HPC data storage systems



Management node(s): Run core system services such as cluster management software, system monitoring software, *batch job scheduler*, etc

ext4

ext4 is the default *local filesystem* for most Linux operating system distributions. It is a journaling file system that can support large storage volumes — up to 1 exbibyte (EiB) — and large single file sizes — up to 16 tebibytes (TiB).

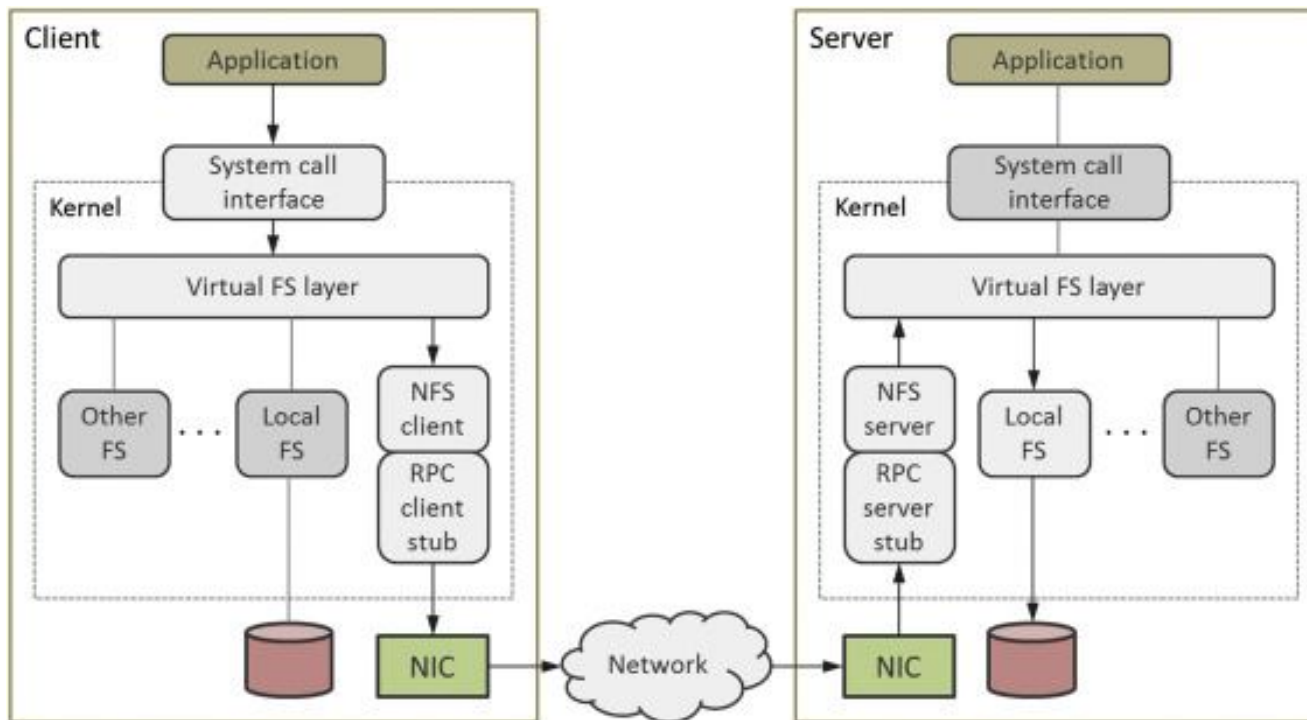
```
mkandes@login01:~$ df -Th | head -n 20
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
devtmpfs	devtmpfs	63G	0	63G	0%	/dev
tmpfs	tmpfs	63G	11M	63G	1%	/run
/dev/sda2	ext4	32G	24G	6.0G	80%	/
tmpfs	tmpfs	63G	14G	50G	22%	/dev/shm
tmpfs	tmpfs	63G	0	63G	0%	/sys/fs/cgroup
/dev/sda1	vfat	100M	0	100M	0%	/boot/efi
/dev/sda4	ext4	32G	13G	17G	43%	/tmp
/dev/sdb1	ext4	879G	44K	834G	1%	/scratch
master:/home	nfs	140G	98G	43G	70%	/exppanse/nfs/mgr1/home
ps-071.sdsc.edu:/ps-data/community-sw	nfs	1.0T	324G	701G	32%	/exppanse/community
10.22.100.114:/pool4/home	nfs	207T	22T	185T	11%	/exppanse/nfs/home4
10.22.100.113:/pool3/home	nfs	199T	22T	177T	12%	/exppanse/nfs/home3
10.22.100.112:/pool2/home	nfs	210T	28T	183T	13%	/exppanse/nfs/home2
10.22.100.111:/pool1/home	nfs	214T	21T	193T	10%	/exppanse/nfs/home1
10.21.0.21:6789,10.21.11.7:6789,10.21.11.8:6789:/	ceph	1.6T	1.2T	467G	71%	/cm/shared
10.22.101.201:/cw3e	nfs	82T	1.0M	82T	1%	/exppanse/nfs/cw3e
192.168.43.5:6789,192.168.43.6:6789:/	ceph	2.0P	159T	1.9P	8%	/exppanse/ceph
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/projects	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/projects
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/scratch	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/scratch

```
[mkandes@login01 ~]$
```

Network File System (NFS)

The Network File System (NFS) is one of the most popular *distributed filesystem* protocols in use on HPC systems. It allows a user on a client computer to access files stored on a remote server over a network.



NFS is often used to mount your **\$HOME** directory system-wide across all nodes of an HPC system.

In this use case, the total amount of storage is typically quite limited and intended only for software installation, batch job scripts, and essential input/output files.

Network File System (NFS)

The [Network File System \(NFS\)](#) is one of the most widely used distributed file system protocols in use on HPC systems. It allows a user on a client computer to access files stored on a remote server over a network.

```
mkandes@login01:~$ df -Th | head -n 20
```

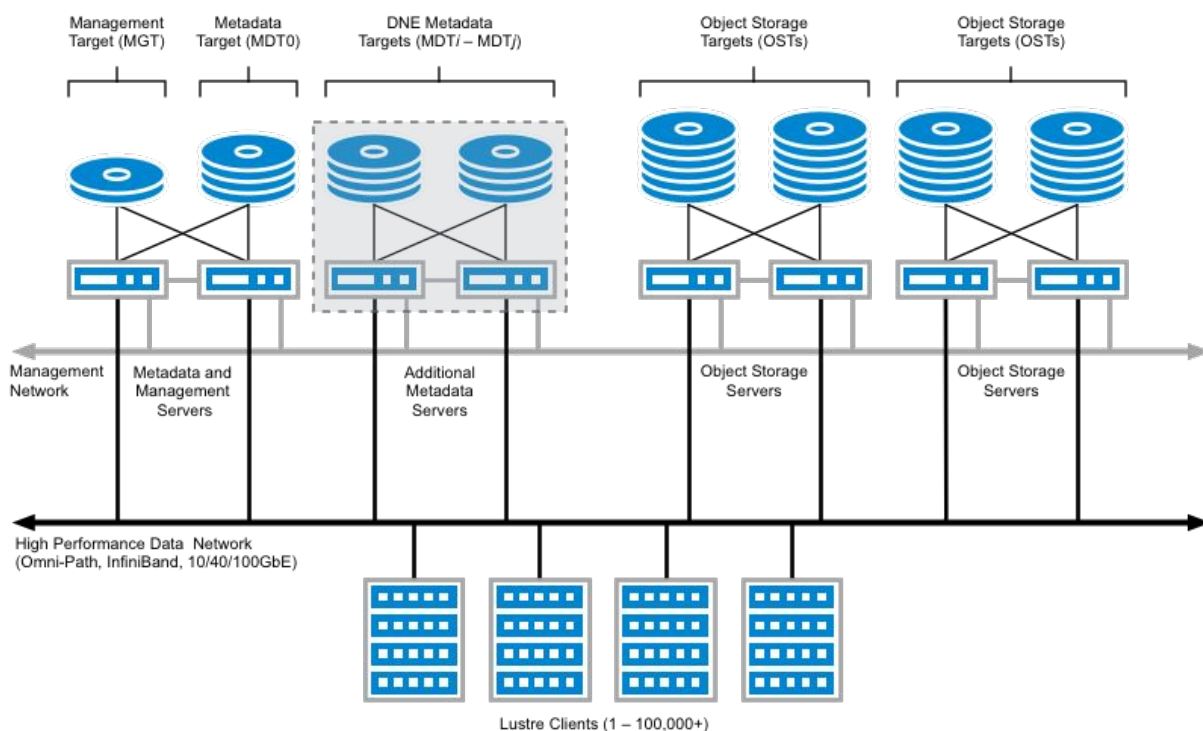
Filesystem	Type	Size	Used	Avail	Use%	Mounted on
devtmpfs	devtmpfs	63G	0	63G	0%	/dev
tmpfs	tmpfs	63G	11M	63G	1%	/run
/dev/sda2	ext4	32G	24G	6.0G	80%	/
tmpfs	tmpfs	63G	14G	50G	22%	/dev/shm
tmpfs	tmpfs	63G	0	63G	0%	/sys/fs/cgroup
/dev/sda1	vfat	100M	0	100M	0%	/boot/efi
/dev/sda4	ext4	32G	13G	17G	43%	/tmp
/dev/sdb1	ext4	879G	44K	834G	1%	/scratch
master:/home	nfs	140G	98G	43G	70%	/exppanse/nfs/mgr1/home
ps-071.sdsc.edu:/ps-data/community-sw	nfs	1.0T	324G	701G	32%	/exppanse/community
10.22.100.114:/pool4/home	nfs	207T	22T	185T	11%	/exppanse/nfs/home4
10.22.100.113:/pool3/home	nfs	199T	22T	177T	12%	/exppanse/nfs/home3
10.22.100.112:/pool2/home	nfs	210T	28T	183T	13%	/exppanse/nfs/home2
10.22.100.111:/pool1/home	nfs	214T	21T	193T	10%	/exppanse/nfs/home1
10.21.0.21:6789,10.21.11.7:6789,10.21.11.8:6789:/	ceph	1.6T	1.2T	467G	71%	/cm/shared
10.22.101.201:/cw3e	nfs	82T	1.0M	82T	1%	/exppanse/nfs/cw3e
192.168.43.5:6789,192.168.43.6:6789:/	ceph	2.0P	159T	1.9P	8%	/exppanse/ceph
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/projects	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/projects
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/scratch	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/scratch

```
[mkandes@login01 ~]$
```

Lustre



Lustre is a *parallel distributed filesystem* commonly used in HPC. It is highly-scalable and can support up to hundreds of thousands of clients, hundreds of petabytes (PB) of storage across hundreds of storage servers, and tens of terabytes per second (TB/s) of aggregate I/O throughput.



Lustre is typically used as a high-performance scratch filesystem, where you can read and write large amounts of data (in parallel; e.g., MPI I/O).

Lustre was not designed for frequent, large bursts of metadata operations. **e.g., reading or writing too many small files too quickly or too often can cause problems filesystem-wide**

Lustre may be a common target source and/or destination file system for your data transfers.

Lustre

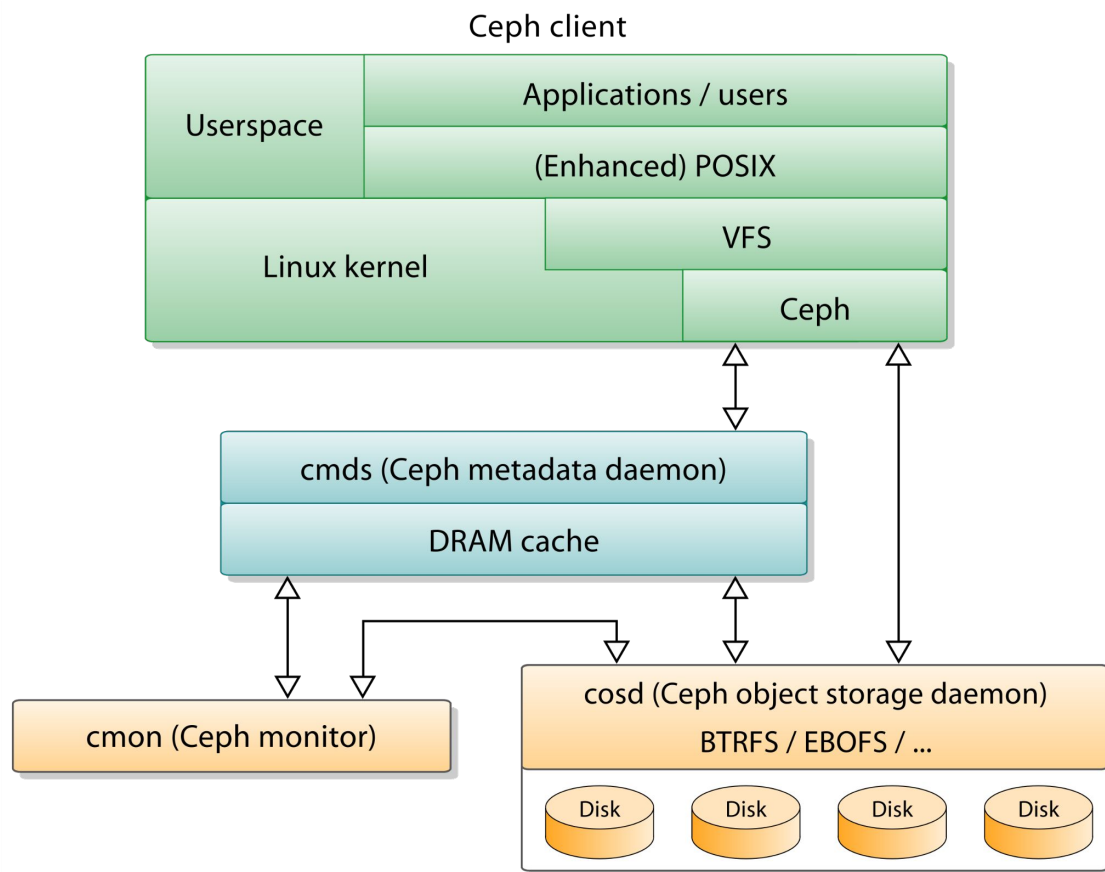


Lustre is a *parallel distributed filesystem* commonly used in HPC. It is highly-scalable and can support up to hundreds of thousands of clients, hundreds of petabytes (PB) of storage across hundreds of storage servers, and tens of terabytes per second (TB/s) of aggregate I/O throughput.

```
mkandes@login01:~  
[mkandes@login01 ~]$ df -Th | head -n 20  
Filesystem                                Type      Size  Used Avail Use% Mounted on  
devtmpfs                                 devtmpfs   63G    0    63G   0% /dev  
tmpfs                                    tmpfs       63G   11M    63G   1% /run  
/dev/sda2                                ext4       32G   24G   6.0G  80% /  
tmpfs                                    tmpfs       63G   14G   50G  22% /dev/shm  
tmpfs                                    tmpfs       63G    0    63G   0% /sys/fs/cgroup  
/dev/sda1                                vfat      100M    0   100M   0% /boot/efi  
/dev/sda4                                ext4       32G   13G   17G  43% /tmp  
/dev/sdb1                                ext4      879G   44K  834G   1% /scratch  
master:/home                             nfs       140G   98G   43G  70% /exppanse/nfs/mgr1/home  
ps-071.sdsc.edu:/ps-data/community-sw     nfs       1.0T  324G  701G  32% /exppanse/community  
10.22.100.114:/pool4/home                  nfs      207T   22T  185T  11% /exppanse/nfs/home4  
10.22.100.113:/pool3/home                  nfs      199T   22T  177T  12% /exppanse/nfs/home3  
10.22.100.112:/pool2/home                  nfs      210T   28T  183T  13% /exppanse/nfs/home2  
10.22.100.111:/pool1/home                  nfs      214T   21T  193T  10% /exppanse/nfs/home1  
10.21.0.21:6789,10.21.11.7:6789,10.21.11.8:6789:/ ceph     1.6T  1.2T  467G  71% /cm/shared  
10.22.101.201:/cw3e                        nfs       82T   1.0M   82T   1% /exppanse/nfs/cw3e  
192.168.43.5:6789,192.168.43.6:6789:/     ceph     2.0P  159T   1.9P   8% /exppanse/ceph  
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/projects lustre    11P   9.1P   1.7P  85% /exppanse/lustre/projects  
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/scratch lustre    11P   9.1P   1.7P  85% /exppanse/lustre/scratch  
[mkandes@login01 ~]$
```



Ceph is a distributed storage system that provides [block](#), [file](#), and [object-based storage](#).



Ceph is typically used as longer-term, large-scale project storage on an HPC system.

Ceph excels at [data replication](#) with [fault tolerance](#) and [high-availability](#), which in turn helps provide strong data durability.

Ceph may be a more common target source and/or destination storage system for your data transfers in the future.

Ceph is a distributed storage system that provides [block](#), [file](#), and [object-based storage](#).

```
mkandes@login01:~$ df -Th | head -n 20
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
devtmpfs	devtmpfs	63G	0	63G	0%	/dev
tmpfs	tmpfs	63G	11M	63G	1%	/run
/dev/sda2	ext4	32G	24G	6.0G	80%	/
tmpfs	tmpfs	63G	14G	50G	22%	/dev/shm
tmpfs	tmpfs	63G	0	63G	0%	/sys/fs/cgroup
/dev/sda1	vfat	100M	0	100M	0%	/boot/efi
/dev/sda4	ext4	32G	13G	17G	43%	/tmp
/dev/sdb1	ext4	879G	44K	834G	1%	/scratch
master:/home	nfs	140G	98G	43G	70%	/exppanse/nfs/mgr1/home
ps-071.sdsc.edu:/ps-data/community-sw	nfs	1.0T	324G	701G	32%	/exppanse/community
10.22.100.114:/pool4/home	nfs	207T	22T	185T	11%	/exppanse/nfs/home4
10.22.100.113:/pool3/home	nfs	199T	22T	177T	12%	/exppanse/nfs/home3
10.22.100.112:/pool2/home	nfs	210T	28T	183T	13%	/exppanse/nfs/home2
10.22.100.111:/pool1/home	nfs	214T	21T	193T	10%	/exppanse/nfs/home1
10.21.0.21:6789,10.21.11.7:6789,10.21.11.8:6789:/	ceph	1.6T	1.2T	467G	71%	/cm/shared
10.22.101.201:/cw3e	nfs	82T	1.0M	82T	1%	/exppanse/nfs/cw3e
192.168.43.5:6789,192.168.43.6:6789:/	ceph	2.0P	159T	1.9P	8%	/exppanse/ceph
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/projects	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/projects
10.22.101.123@o2ib:10.22.101.124@o2ib:/exppanse/scratch	lustre	11P	9.1P	1.7P	85%	/exppanse/lustre/scratch

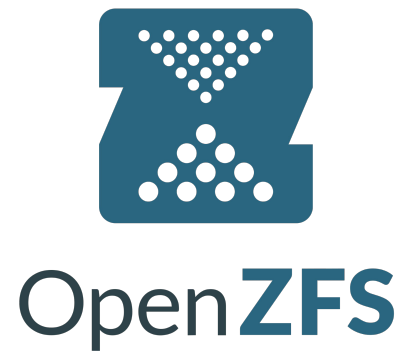
```
[mkandes@login01 ~]$
```

Other Data Storage and File Systems

- [BeeGFS](#)
- [DAOS](#)
- [IBM Spectrum Scale \(GPFS\)](#)
- [OrangeFS](#)
- [Qumulo](#)
- [VAST](#)
- [WEKA](#)
- [ZFS](#)



IBM
**Spectrum
Scale**



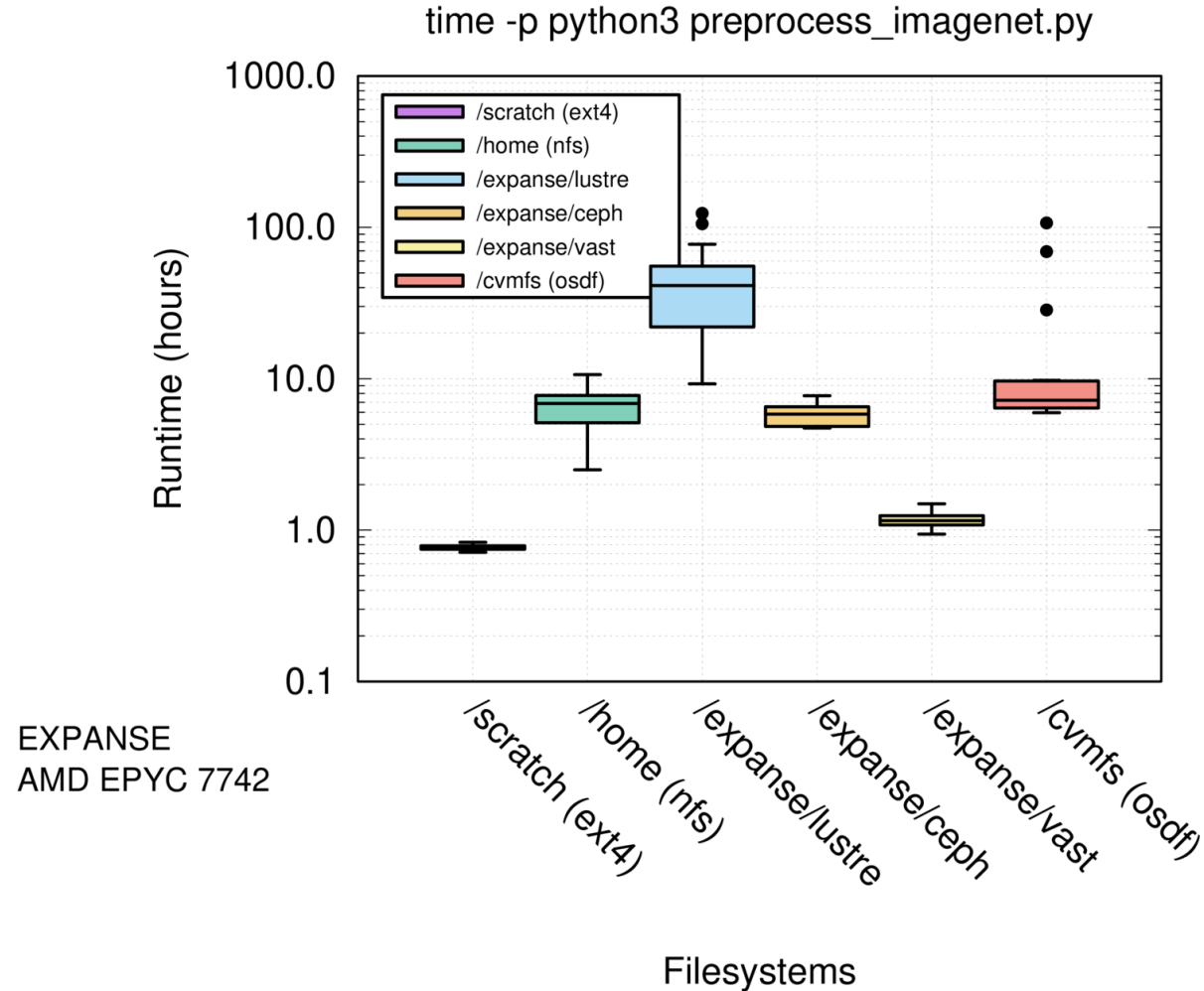
<https://io500.org>



Table of Contents

- What is a filesystem?
- Data storage devices and systems
- Files, Directories, and Inodes
- Filesystems in High-Performance Computing
- **Filesystem performance: An AI example**
- Summary and Conclusion: Best practices
- Questions & Answers (30 min)

Transforming ImageNet to TFRecords



<https://github.com/sdsc-complecs/data-management/blob/main/run-preprocess-imagenet-tensorflow-2.8.3-ubuntu-20.04-cuda-11.2-mlnx-ofed-4.9-4.1.7.0-openmpi-4.1.3-20221008-lustre-scratch.sh>

Table of Contents

- What is a filesystem?
- Data storage devices and systems
- Files, Directories, and Inodes
- Filesystems in High-Performance Computing
- Filesystem performance: An AI example
- **Summary and Conclusion: Best practices**
- Questions & Answers (30 min)

Summary and Conclusion: Best practices

- Understand what types of filesystems are available to you on an HPC system
- Think about how reading and writing (and packaging) your data may affect overall performance of your workload when using any given filesystem
- Avoid small file reads and writes, if possible.
- Keep reads and writes as infrequent as possible
- Always try and use node-local /scratch disks when available.
- Don't have too many batch jobs accessing the same files simultaneously; make local copies on node-local /scratch instead
- Keep the number of files (and directories) in each directory reasonable (< 10K)
- **Do not colorize your SHELL! Do you want more metadata lookups!?**
- **Always backup important data to your own storage; HPC file systems don't typically have a backup of your data**

Table of Contents

- What is a filesystem?
- Data storage devices and systems
- Files, Directories, and Inodes
- Filesystems in High-Performance Computing
- Filesystem performance: An AI example
- Summary and Conclusion: Best practices
- Questions & Answers (30 min)

Additional Information

- [Data Management and File Systems on Expanse](#)
- [Using the NIH HPC Storage Systems Effectively](#)



Questions & Answers

